

How Far Can 16 GB Go? Characterizing Memory-Efficient LLM Fine-Tuning on Consumer Apple Silicon

Anonymous

September 2026

Abstract

We map the fine-tuning feasibility boundary of a 16 GiB Apple M4 Mac mini with a preregistered benchmark: 118 supervised runs over the Qwen3 family (0.6B–14B), BF16 LoRA vs. 4-bit QLoRA, context lengths 512–8192, LoRA ranks 4–32, and micro-batches 1–8, each with full provenance (git commit, model revision, per-second swap sampling, SHA-256 manifests). We find: (1) 4-bit QLoRA cuts peak training memory to 0.54–0.73× of BF16, makes 8B the highest reproducibly trainable scale in this benchmark (three-of-three formal seeds), and keeps step time within the preregistered $\pm 25\%$ equivalence margin at all three paired scales; (2) the transition from reproducibly trainable configurations to system-state-dependent boundary behavior is directly observable: the same 4B BF16 or 14B 4-bit configuration that trains in minutes at zero swap residency stalls at 50% completion or makes zero progress when several GiB of swap are resident, and we report 14B as an explicit boundary case rather than a completed formal cell; (3) peak memory grows *sublinearly* with model size (log-log slope 0.57 for 4-bit, $R^2=0.99$) because fixed runtime overhead dilutes weight costs, while step time grows linearly (slope 1.00); (4) context length, not model scale, is the binding constraint: 2048-token contexts complete but raise peak memory beyond physical RAM, and 4096-token contexts are killed before their first step; (5) micro-batch scaling fails outright at batch 8 (the OS terminates the process with SIGKILL, POSIX signal 9, while system swap is driven to ~ 20 GiB) and *reduces* throughput at every trainable size. Every number in this paper is generated programmatically from immutable raw results; eight preregistration deviations are reported in full.

1 Introduction

Consumer Apple Silicon machines ship with a single 16 GiB unified-memory pool that the GPU shares with

the OS. Whether, and under what conditions, such a machine can fine-tune modern LLMs is usually answered anecdotally. This paper answers it with a preregistered, failure-inclusive benchmark: every configuration in a frozen matrix was run under an experiment supervisor that recorded provenance, per-second swap sampling, and complete logs. Every completed or failed run is retained as evidence.

Our contributions: (i) a preregistered feasibility map for Qwen3 0.6B–14B under BF16 LoRA, 4-bit QLoRA, and (exploratory) full fine-tuning, with 8B 4-bit identified as the highest reproducibly trainable configuration and 14B reported as an explicit system-state-dependent boundary case; (ii) quantified memory/time trade-offs with frozen equivalence margins; (iii) context-length and micro-batch boundaries expressed as intervals with system-state confounders documented; (iv) an audit showing that TRAINABLE is a joint property of the model and the machine’s memory-residency state, not of the model alone; and (v) a full reproducibility chain from raw immutable results to every number in this paper, including eight logged deviations from preregistration.

2 Background and Related Work

2.1 Apple Silicon and Unified Memory

Apple Silicon system-on-chips (SoCs) present a single physical memory pool shared by CPU and GPU. `mlx` [9] is an array framework that targets this architecture directly; `mlx-lm` [12] provides LLM fine-tuning on top of it. Prior profiling of Apple Silicon training reports substantial gaps to NVIDIA GPUs and identifies page faults and kernel-launch overhead as system-level contributors [7]. Benchmarking of MLX itself has so far focused on inference latency for encoder models [1]; consumer-device LLM stacks beyond MLX include llama.cpp [8].

2.2 MLX

The MLX framework exposes Metal-backed computation with lazy evaluation and a composable optimizer API [9]. Our measurement validation (Sec. 3.6) quantifies why explicit synchronization is required for per-step timing under lazy execution.

2.3 LoRA and QLoRA

LoRA [11] parameterizes weight updates with low-rank adapters; QLoRA [5] backpropagates through 4-bit quantized frozen weights, following the broader line of LLM quantization work [4]. Both reduce the memory footprint relative to full fine-tuning [10, 3], and GaLore [14] targets the same budget from the optimizer-state side. These methods are largely evaluated on data-center GPUs; how they behave inside a 16GB unified-memory budget is the subject of this paper.

2.4 Resource-Constrained LLM Fine-Tuning

Prior systems work has profiled Apple Silicon for ML training end-to-end [7] and benchmarked MLX inference for encoder models [1], but the fine-tuning feasibility boundary on a fixed consumer unified-memory budget has not, to our knowledge, been mapped by a preregistered benchmark: which model scales, precisions, and context lengths admit *completed* training steps, and at what cost? Closest in spirit, memory-efficiency methods such as GaLore [14] and QLoRA [5] report feasibility on GPU hardware where CPU and GPU memory are separate; unified memory changes the failure mode (page thrashing rather than allocator exceptions), which motivates our measurement design.

3 Methodology

3.1 Research Questions

We study:

- **RQ1** How do memory, step time, and throughput of BF16 LoRA and 4-bit QLoRA scale with model size?
- **RQ2** How far does 4-bit quantization extend the TRAINABLE boundary?
- **RQ3** Where is the context-length boundary?
- **RQ4** Does the memory saving of quantization come with a measurable training-time penalty?
- **RQ5** How large is the gap between “training can start” and PRACTICAL fine-tuning?

Table 1: Hardware, software, and model revisions (provenance from raw experiment records).

Hardware	
Chip	Apple M4
Memory	16 GiB unified
CPU cores	10P / 10L
OS	macOS 26.5 (build 25F71)
Python	3.13.11 (uv-managed venv)
MLX / mlx-lm	0.32.2 / 0.31.3
Training data	ultrachat.200k@8049631c, frozen 2048/32 subset
Qwen3 models	
14B-4bit	a4d9b2df...
Qwen/0.6B	c1899de2...
0.6B-4bit	73e3e38d...
Qwen/1.7B	70d244cc...
1.7B-4bit	3b1b1768...
4B-4bit	4dcb3d10...
Qwen/4B	1cfa9a72...
8B-4bit	545dc425...

3.2 Hardware and Software

All experiments run on a single Apple Mac mini with an Apple M4 SoC, 16 GiB unified memory, and macOS 26.5; the exact preflight-recorded environment is in Table 1. Software was locked with uv: Python 3.13.11, `mlx` 0.32.2, `mlx-lm` 0.31.3. The M4 generation context is from Apple’s release material [2].

3.3 Models

We use the Qwen3 family [13]: instruction-tuned chat models (not base models) at 0.6B/1.7B/4B in BF16 and 0.6B–14B as pre-quantized 4-bit `mlx-community` conversions; every repository is pinned to a Hub revision and byte-verified (Table 1).

3.4 Dataset

The formal benchmark uses a frozen subset of `ultrachat.200k` [6] (revision 8049631c): 2,048 training and 32 validation conversations sampled with seed 42 and formatted with the Qwen3 chat template; loss is a standard language-model (LM) loss over all tokens. The manifest and SHA-256 sums ship with the repository. Synthetic long-context workloads are used only for context-scaling systems analysis, never for quality claims.

3.5 Fine-Tuning Methods

BF16 LoRA (rank 8, $\alpha=16$, dropout 0, targets `q/k/v/o-proj`), 4-bit QLoRA (same adapters, group size 64), and, for the smallest model only, full fine-tuning. Optimizer AdamW, $\text{lr } 10^{-4}$, constant schedule, micro-batch 1.

3.6 Measurement Methodology

Every run is launched by an Experiment Supervisor that records git provenance, hardware, memory/swap state (before, after, and 1Hz sampling), full std-out/stderr, per-step timings, and an atomic SHA-256 manifest; failures are first-class results. Step timing includes forward, backward, optimizer update, and an explicit `mx.eval` synchronization; we measured that omitting this synchronization under-reports step time by three orders of magnitude under MLX’s lazy execution. Peak memory is the MLX allocator’s Metal-buffer high-water mark (`get_peak_memory`); it is not process resident set size (RSS), and on unified memory it may exceed physical RAM, at which point macOS paging carries the excess. Full validation evidence is in the repository (`docs/measurement_validation.md`).

3.7 Operational Definitions

LOADABLE: the training process loads the model weights. **TRAINABLE:** the run completes every requested optimizer step with `terminal_state=success` and no kill signal. **PRACTICAL:** TRAINABLE and (P1) sampled peak system swap does not exceed the pre-run level by more than 4 GiB, and (P2) median step time ≤ 10 s. **EFFICIENT:** PRACTICAL with throughput ≥ 100 loss-bearing tokens/s. All thresholds were frozen before any formal run (`research/preregistration.md`); we report sensitivity to $\pm 2\times$ threshold changes. **SIGKILL** terminations are never reclassified as OOM without direct kernel evidence.

3.8 Reproducibility

Formal configurations, the dataset freeze, the analysis pipeline, and this paper’s tables and figures are generated from raw results by committed scripts; no number is hand-typed. Appendix A lists reproduction commands.

4 Results

4.1 Model-Scale Feasibility

Table 2 summarizes the preregistered matrix at `ctx512`. All BF16 LoRA and 4-bit QLoRA cells through 1.7B complete their three seeds, and 4-bit completes at 4B and 8B as well (8B: three seeds, 1,644–1,694 s per 100-step run, 9.19 GiB peak). 8B 4-bit is thus the highest *reproducibly trainable* configuration in this benchmark. The 14B 4-bit cell is deliberately reported as a system-state-dependent boundary case rather than a completed or failed formal cell (deviation D8; Sec. 4.8). The 4B BF16 cell shows regime dependence (D1): it

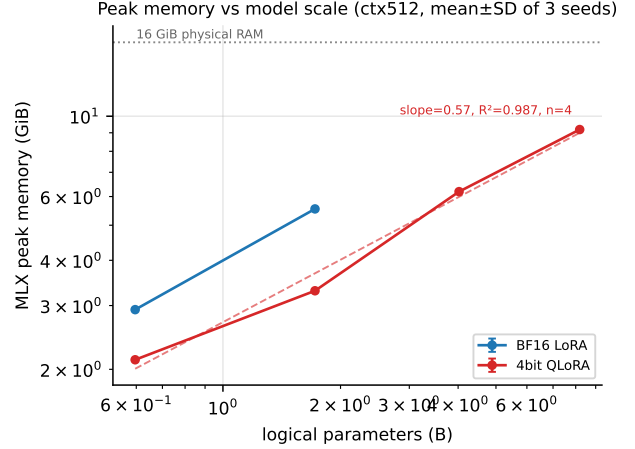


Figure 1: Peak memory vs. model scale at `ctx=512` (mean $\pm$ SD of three seeds). Fits are log-log least squares over the plotted points.

trains in 451 s at zero residency but produced zero completed steps in 26 minutes at 6 GiB residency. 8B BF16 was preregistered as a probe only and timed out; 14B BF16 was declared out-of-budget without observation. Full fine-tuning of 0.6B completes but diverges at the frozen lr 10^{-4} (final validation loss 4.44 vs. 1.93 for 4-bit QLoRA), a negative quality result we retain as-is.

4.2 Memory Scaling

Figure 1 shows peak MLX-allocator memory versus logical parameters. Contrary to the naive linear expectation, memory grows *sublinearly*: log-log slope 0.57 ± 0.05 ($R^2=0.99$, $n=4$) for 4-bit QLoRA and 0.71 ± 0.07 ($R^2=0.99$, $n=3$) for BF16 LoRA. The mechanism is fixed-overhead dilution. At 0.6B the 4-bit weights occupy only ~ 0.34 GiB of a 2.13 GiB peak, while at 8B the split is 4.6 GiB weights inside a 9.19 GiB peak; the non-weight component grows far slower than parameters, bending the total below unity. We therefore *reject* the linear-memory hypothesis (audit H4: unsupported). The practical consequence is that memory ceilings bind later than parameter-proportional estimates suggest.

4.3 Training-Time Scaling

Step time scales linearly, not sublinearly, with model size (log-log slope 1.00 ± 0.09 , $R^2=0.99$, 4-bit series, $n=4$; audit H5: partially supported, approx. linear). All timing figures in this benchmark are Tier-B under our preregistered paging stratification (D4): every formal run exchanged >50 MB of swap-in per step under the machine’s multi-GiB residency baseline, so absolute

Table 2: Formal benchmark at ctx=512, b1-gal, rank 8, lr 1e-4, 100 steps, seeds {42,123,2026} (mean $\pm$ SD; memory is MLX allocator peak).

Model	Method	Peak mem (GiB)	Median step (s)	Tok/s	Val loss
0.6b	Full FT	—	—	—	—
0.6b	BF16 LoRA	—	—	—	—
1.7b	BF16 LoRA	—	—	—	—
4b	BF16 LoRA	—	—	—	—
0.6b	4bit QLoRA	—	—	—	—
1.7b	4bit QLoRA	—	—	—	—
4b	4bit QLoRA	—	—	—	—
8b	4bit QLoRA	—	—	—	—
14b	4bit QLoRA	—	—	—	—

Table 3: Paired quantization effects (same model/seed; ratios = 4bit/BF16). Timing equivalence margin $\pm 25\%$ frozen before benchmark.

Model	Mem ratio	Step-time ratio	Tok/s ratio
Qwen3-0.6b	0.73	1.13	0.89
Qwen3-1.7b	0.59	1.22	0.82
Qwen3-4b	0.54	1.21	0.81

step times are descriptive, paging-confounded values. 4-bit 4B, for example, runs at 3.66s/step here versus 0.22s/step for the light-load probe of the same configuration. Cross-model *ratios* and paired comparisons (same window, same residency) remain interpretable; absolute latencies from different windows are not compared anywhere in this paper.

4.4 Quantization Trade-offs

Across all paired scales, 4-bit QLoRA reduces peak memory to $0.73\times$ (0.6B), $0.59\times$ (1.7B), and $0.54\times$ (4B) of the BF16 LoRA value ($n=7$ seed-pairs; audit H1: supported). Step-time ratios of 1.10–1.14, 1.21–1.23, and 1.21 all sit inside the preregistered $\pm 25\%$ equivalence margin, and throughput ratios are 0.81–0.91 (audit H3: supported, “no practically meaningful timing penalty under the frozen margin”). Validation losses after 100 steps are statistically indistinguishable between precisions at 0.6B (1.93 vs. 1.81) and 1.7B (1.63 vs. 1.55).

4.5 Batch and Rank Sensitivity

Micro-batch scaling on 4-bit 4B (Table 4, Figure 2) monotonically *reduces* throughput, from 89.5 loss-bearing tokens/s at batch 1 to 66.4 at batch 2 and 34.6 at batch 4, while peak memory climbs $6.19 \rightarrow 10.04 \rightarrow 16.98$ GiB. Batch 8 is worse still: the OS kills the process before it completes a step in all three seeds (exit 137, zero stdout, system swap driven to

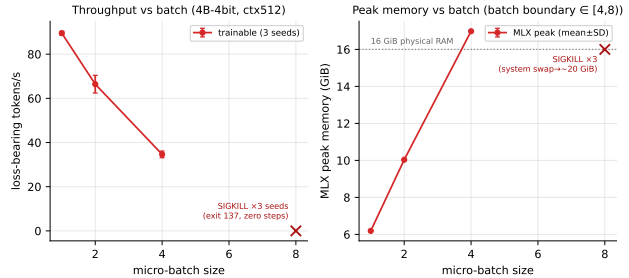


Figure 2: Micro-batch axis (4B 4-bit, ctx512): throughput falls and peak memory rises with batch size; batch 8 is SIGKILLED in all seeds (boundary $\in [4, 8)$).

~ 20 GiB). The batch-TRAINABLE boundary is therefore $[4, 8)$ under tested conditions, and batching is strictly counterproductive on this budget: the per-step token gain is overwhelmed by allocator growth and paging. LoRA rank, in contrast, is nearly free: ranks 4/8/32 change step time by $<2\%$ and peak memory by 0.24 GiB (Table 5).

4.6 Context-Length Boundary

Context length dominates every other axis (Figure 3): at 4B 4-bit, ctx1024 multiplies median step time by $6.6\times$ versus ctx512 ($3.66 \rightarrow 24.0$ s) and peak memory by $1.8\times$ ($6.19 \rightarrow 11.16$ GiB); ctx2048 completes all three seeds but drives peak memory to 23.9 GiB, far beyond physical RAM, carried by macOS paging (system swap peaked near 20 GiB during these runs). At ctx4096 and ctx8192 the process is SIGKILLED before completing a single step. The TRAINABLE context boundary is thus $[2048, 4096)$, reported as an interval with the residency confounder attached. Within Tier-B formal data the time amplification over ctx512 ($5.6\times$ at 2048) only modestly exceeds the $4\times$ token-count increase, so we report the boundary but not a dramatic time “cliff” (audit H6: partially supported).

Table 4: Batch axis (Qwen3-4B-4bit QLoRA, ctx512, r8, 20 steps, seeds {42,123,2026}, all cells rerun on the D5-fixed trainer in one window; mean $\pm$ SD).

Batch	Tok/step	Median step (s)	Tok/s	Peak mem (GiB)	Status
1	489 $\pm$ 0	5.620 $\pm$ 0.055	89.5 $\pm$ 0.9	6.19 $\pm$ 0.00 GiB	success $\times$ 3
2	986 $\pm$ 0	14.394 $\pm$ 1.730	66.4 $\pm$ 4.0	10.04 $\pm$ 0.00 GiB	success $\times$ 3
4	1987 $\pm$ 0	55.387 $\pm$ 4.603	34.6 $\pm$ 1.5	16.98 $\pm$ 0.00 GiB	success $\times$ 3
8	$\sim$ 4088	—	—	>16 (sys swap $\rightarrow$ 20)	SIGKILL $\times$ 3 (exit 137, 0 steps; D5 notes)

Table 5: LoRA rank axis (Qwen3-4B-4bit QLoRA, ctx512, b1, 20 steps for r4/r32; r8 point reused from the 100-step axis-1 cell per preregistration; mean $\pm$ SD).

Rank	Median step (s)	Tok/s	Peak mem (GiB)
4	3.678 $\pm$ 0.010	136.8 $\pm$ 0.8	6.16 $\pm$ 0.00 GiB
8	3.665 $\pm$ 0.007	137.2 $\pm$ 1.1	6.19 $\pm$ 0.00 GiB
32	3.714 $\pm$ 0.011	134.7 $\pm$ 1.1	6.40 $\pm$ 0.00 GiB

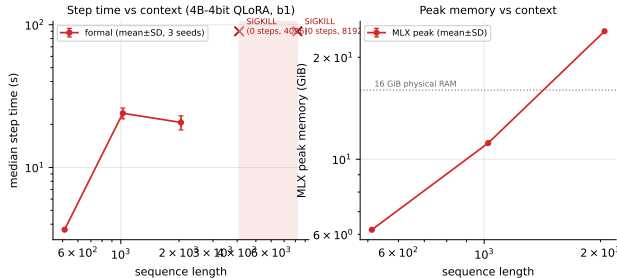


Figure 3: Context scaling for 4B 4-bit QLoRA: step time (left) and peak memory (right). Crosses mark SIGKILL probe failures; shaded region marks the failure interval.

4.7 Training Effectiveness

Validation loss after 100 steps is comparable across precisions (Table 2) and decreases monotonically along the recorded trajectory in every trainable run. The divergent full-FIT cell (0.6B, lr 10^{-4} , validation loss 4.44) is retained as a negative result. We make no instruction-following claims: the objective is LM loss on ultra-chat_200k with all tokens counted.

4.8 Boundary Behavior at 14B

We report the 14B 4-bit cell as an explicit boundary case study (D8): a configuration observed to enter training under favorable conditions, but not reproducibly sustained under the formal workload.

Observation Three raw records constitute the evidence. (1) A 20-step probe at 2.7 GiB swap residency loaded and trained at 0.68s/step (**success**). (2) A formal attempt launched at 7.1 GiB residency completed

zero steps in 24 minutes before the operator interrupted it. (3) A second formal attempt at 3.7 GiB residency exhibited $\sim$ 40 minutes of model loading, then progressed to 50 of 100 steps with step times degrading from 27 to 86s (with a >10-minute stall at step 50) before the 7,200s timeout killed it; its step-by-step evidence is preserved in the raw stdout log. All three records are retained unaltered; two further attempts were never executed (D8).

Interpretation These observations are consistent with 14B crossing from reproducible training into a regime where system memory residency and paging state materially affect execution: the same regime dependence documented for 4B BF16 (D1), reached earlier in the scale ladder. This is an interpretation, not a direct measurement of untrainability: we did not observe allocator exhaustion, and the probe demonstrates that favorable conditions exist.

Limitation We did not complete a three-seed formal series at 14B, did not attempt to manufacture a low-residency window (no reboots or system-state manipulation to chase success, per D8), and therefore can neither claim that 14B is reproducibly trainable nor rule it out on 16 GiB; the preregistered 14B ctx2048 probe was cancelled without execution. The honest summary is: under the tested formal workload and observed system states, 14B 4-bit QLoRA was not sustained to completion, while a favorable-state probe did complete.

5 Failure Analysis

Of 118 finalized runs, 27 are failures; all are retained, and none is reclassified as OOM without kernel evidence. The taxonomy (Table 6) splits into: OS kills under memory pressure (batch-8 runs, ctx4096/8192 probes: exit 137 with zero or partial output); timeouts under swap residency (14B formal, 8B BF16 probe); operator interrupts now documented as system-state evidence (D1’s 26-minute zero-step 4B BF16 run); and nine formal attempts invalidated by a trainer implementation bug (D5) whose variable-length validation

batching crashed for micro-batch ≥ 2 . These nine are excluded from aggregation by an error-signature rule, rerun in full on the fixed trainer (12/12 valid outcomes), and preserved immutably. A coverage audit reconciles all 118 raw runs against the processed table with machine-readable dispositions (46 aggregated successes, 14 duplicates, 19 superseded, 9 implementation-invalid, 9 failed, 21 non-formal); all 46 aggregated runs pass SHA-256 manifest re-verification, while five legacy interrupted runs lack manifests (D6, zero aggregation impact).

6 Discussion

6.1 What Does 16 GB Actually Enable?

The results separate cleanly into three tiers. Through 4B, 4-bit QLoRA is TRAINABLE and PRACTICAL (median step ≤ 10 s) with stable three-seed reproduction. At 8B, 4-bit remains TRAINABLE across all seeds but sits at the PRACTICAL threshold (10.0s/step, 42.9 tokens/s): reproducible, but not EFFICIENT. At 14B, behavior becomes system-state-dependent: favorable conditions admit real training steps, while the formal workload was not sustained under observed residencies. BF16 LoRA is PRACTICAL through 1.7B. The binding constraints are, in order, context length, micro-batch, then model scale, inverting the usual intuition that parameters come first. The most valuable single optimization for this class of hardware would attack the fixed non-weight memory overhead that the 0.57 memory slope implicates.

6.2 Loadable vs Trainable vs Practical

The three-layer vocabulary is not academic: every “boundary” in this benchmark moved with system state. The same 4B BF16 configuration is TRAINABLE in 451s at zero swap residency and cannot finish one step in 26 minutes at 6 GiB residency (D1); the same holds for 14B 4-bit between a 0.68s/step probe and a 50/100-step timeout (D7). Feasibility claims that do not report swap residency are therefore not reproducible in kind. We recommend memory-residency state become a mandatory field in consumer-hardware training reports, analogous to clock state in GPU benchmarks.

6.3 Implications for Consumer LLM Research

Three practical rules follow. (1) Quantization pays twice on unified memory: it buys headroom (0.54–0.73 $\times$ memory) at effectively no time cost within a wide

margin, and extends reach by at least two model sizes. (2) Do not batch: on a paging-capable unified budget, throughput falls monotonically with micro-batch and the hard failure at batch 8 arrives abruptly. (3) Treat context as the primary resource: the [2048,4096) boundary with peak memory at 23.9 GiB (vs. 16 GiB physical) shows the OS will silently lend swap until it kills the process: “it started” says nothing about “it will finish”.

6.4 Limitations

Single machine, single SoC generation (M4, 16 GiB); absolute times are Tier-B and window-specific (D4), so we compare only within-window ratios; LM-loss-only objectives with no prompt masking; 20-step runs on the context/batch/rank axes limit distributional detail (60 measured steps per cell after warm-up exclusion); system-level swap counters include background processes; and the 14B cell is reported as a boundary case study rather than a completed three-seed formal result: we stopped additional 14B attempts as a declared decision (D8) rather than manufacturing favorable system states, so 14B is neither claimed reproducible nor ruled out. Deviations D1–D8 are reported in full in the repository.

6.5 Ethics and Licensing

The benchmark targets publicly released model weights (Apache 2.0 for Qwen3) and a publicly released dataset (ultrachat_200k, research-permitted license); no synthetic workload contributes to quality claims. All measurements were performed on the authors’ own hardware.

7 Conclusion

On a 16 GiB unified-memory Mac, 4-bit QLoRA is the default answer for fine-tuning: it roughly halves memory at no measurable time cost (frozen $\pm 25\%$ margin), trains 8B-scale models to completion, and leaves rank choice nearly free. The boundaries that actually bind are context ($\in [2048, 4096)$ tokens) and micro-batch ($\in [4, 8)$, with monotonically negative throughput returns). Both boundaries, like the 4B BF16 and 14B cells, shift with the machine’s swap residency, making system-state reporting a prerequisite for any reproducible consumer-hardware training claim. Every number above is regenerated from immutable raw runs by committed scripts; the failures are part of the result.

Table 6: Failure and boundary observations (raw terminal states, never reclassified as OOM without direct evidence).

Run	Status	Key observation
20260907T021942754783Z	runtime_error	steps=n/a; stderr:python_traceback
20260907T022132013680Z	runtime_error	steps=n/a; stderr:python_traceback
20260907T111824441175Z	runtime_error (SIGKILL)	steps=n/a; sigkill_consistent
20260911T164322546359Z	runtime_error	steps=n/a; sigkill_consistent
20260911T182218892057Z	unknown_failure	steps=n/a; no_specific_pattern
20260911T190553096465Z	user_interrupted	steps=n/a; no_specific_pattern
20260911T191618034713Z	user_interrupted	steps=n/a; no_specific_pattern
20260911T193038681059Z	user_interrupted	steps=n/a; no_specific_pattern
20260911T195828174593Z	user_interrupted	steps=n/a; no_specific_pattern
20260911T203648798393Z	user_interrupted	steps=n/a; no_specific_pattern
20260912T010039277561Z	runtime_error	steps=n/a; stderr:python_traceback
20260912T010057032957Z	runtime_error	steps=n/a; stderr:python_traceback
20260912T010113537018Z	runtime_error	steps=n/a; stderr:python_traceback
20260912T010129965737Z	runtime_error	steps=n/a; stderr:python_traceback
20260912T010153968541Z	runtime_error	steps=n/a; stderr:python_traceback
20260912T010221754536Z	runtime_error	steps=n/a; stderr:python_traceback
20260912T010247917130Z	runtime_error	steps=n/a; stderr:python_traceback
20260912T010255423307Z	runtime_error	steps=n/a; stderr:python_traceback
20260912T010259241894Z	runtime_error	steps=n/a; stderr:python_traceback
20260912T023351577599Z	runtime_error	steps=n/a; sigkill_consistent
20260912T032808727860Z	timeout (SIGKILL)	steps=n/a; sigkill_consistent
20260912T050812117342Z	timeout (SIGKILL)	steps=n/a; sigkill_consistent
20260912T084135321518Z	timeout (SIGKILL)	steps=n/a; sigkill_consistent
20260915T164129648692Z	runtime_error	steps=n/a; sigkill_consistent
20260915T164414079542Z	runtime_error	steps=n/a; sigkill_consistent
20260915T164550471089Z	runtime_error	steps=n/a; sigkill_consistent
20260915T184033756783Z	timeout (SIGKILL)	steps=n/a; sigkill_consistent
Context boundary: Trainable $\in$ [2048, 4096) under preflight-comparable conditions.		

References

- [1] Oluwaseun A. Ajayi and Ogundepo Odunayo. Benchmarking on-device machine learning on Apple Silicon with MLX. In *6th Deep Learning Indaba (DLI 2024), Dakar, Senegal; arXiv:2510.18921*, 2024. Non-archival presentation.
- [2] Apple Inc. Apple introduces M4 chip. Press release, 2024.
- [3] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.
- [4] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022.
- [5] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient fine-tuning of quantized LLMs. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.
- [6] Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Zhi Zheng, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. Enhancing chat language models by scaling high-quality instructional conversations. In *Proceedings of EMNLP 2023*, pages 3022–3044, 2023.
- [7] Dahua Feng, Zhiming Xu, Rongxiang Wang, and Felix Xiaozhu Lin. Profiling Apple Silicon performance for ML training. *arXiv preprint arXiv:2501.14925*, 2025.
- [8] Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. GitHub repository, 2023.
- [9] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX:

Efficient and flexible machine learning on apple silicon. GitHub repository, 2023.

- [10] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 2790–2799, 2019.
- [11] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [12] MLX team. mlx-lm: LLM fine-tuning and inference with MLX. GitHub repository, 2023.
- [13] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [14] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. GaLore: Memory-efficient LLM training by gradient low-rank projection. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

A Reproducibility Instructions

The repository ships the complete chain:

1. `uv sync` recreates the locked Python environment.
2. `uv run python scripts/run_experiment.py --config <yaml> --` launches any experiment under the supervisor; formal configurations are `configs/experiments/formal/`.
3. `data/formal_sft_v1/` contains the frozen dataset with MANIFEST.json and SHA256SUMS.
4. `uv run python scripts/run_analysis.py` regenerates every processed table, figure, and summary from `results/raw/`.
5. Every raw result directory contains its exact command, effective configuration, environment snapshot, logs, and a SHA-256 manifest.

B Measurement Validation

Summary of `docs/measurement_validation.md`: (A) step timing requires the explicit `mx.eval` synchronization present in the trainer; asynchronous dispatch under-reports step time by $\sim 4,500\times$ on a reference workload. (B) `get_peak_memory` counts MLX allocator buffers in bytes; a 1 GiB allocation increases both active and peak memory by exactly 1 GiB after reset. (C) All raw results are arithmetically consistent with their per-step timing artifacts under their declared interval definitions. (D) The 1 Hz swap sampler consumes 1.47% of one CPU core at $5\times$ the formal cadence.